LAB # 06

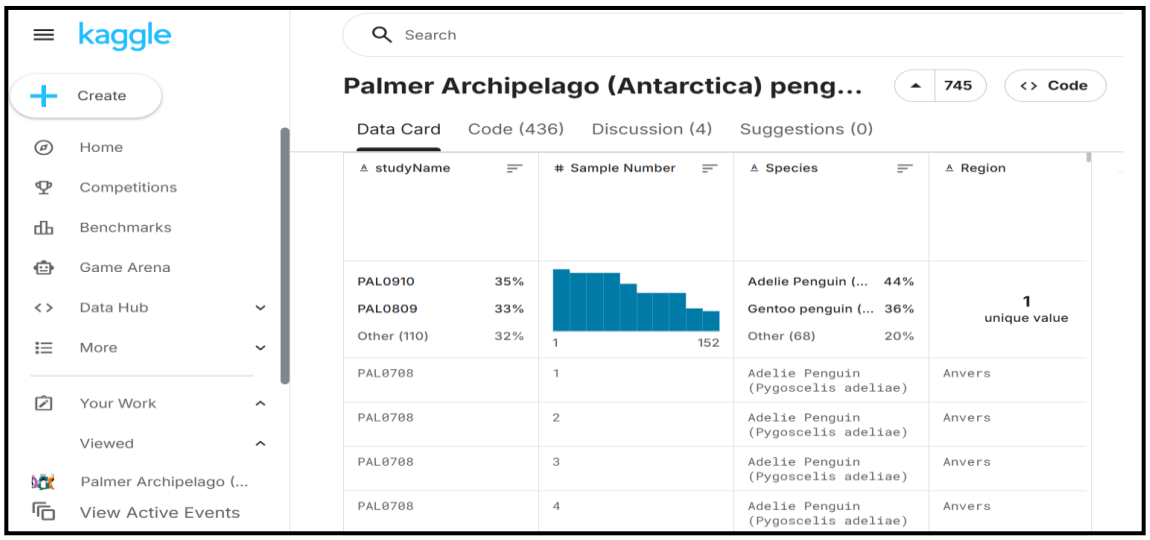
OBJECTIVE:

To implement a neural network (Multilayer Perceptron) using TensorFlow/Keras and apply it to classify handwritten digits (0–9) using the MNIST dataset.

LAB TASK:

Download a dataset containing three input features and three output classes. Load and preprocess the dataset by handling missing values and encoding the target variable for multi-class classification. Build a MLP model using an Ann. Train the model on the dataset and evaluate its performance using appropriate metrics. Visualize the dataset using a three-dimensional (3D) plot to better understand the distribution of the classes and analyze how different features contribute to the classification and examine the decision boundaries of the model.

KAGGLE DATASET:



CODE:

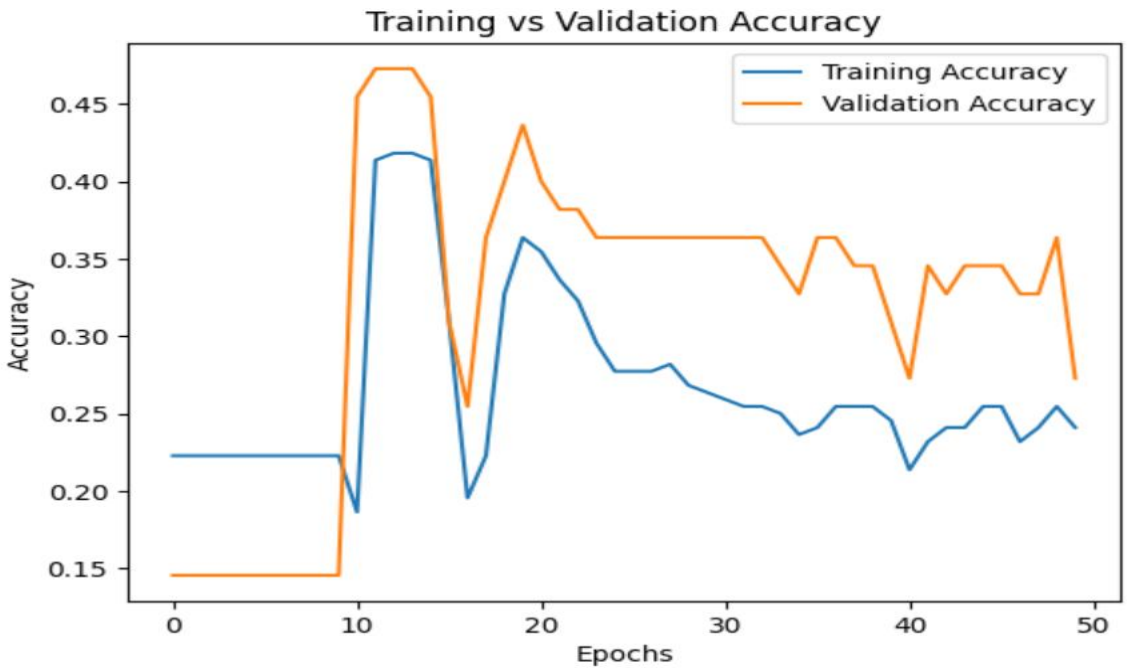
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from mpl_toolkits.mplot3d import Axes3D
data = pd.read_csv("penguins_size.csv")
print("First 5 rows of dataset:")
print(data.head())
X = data[['culmen_length_mm',
          'culmen_depth_mm',
          'flipper_length_mm']]
y = data['species']
imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
model = Sequential()
```

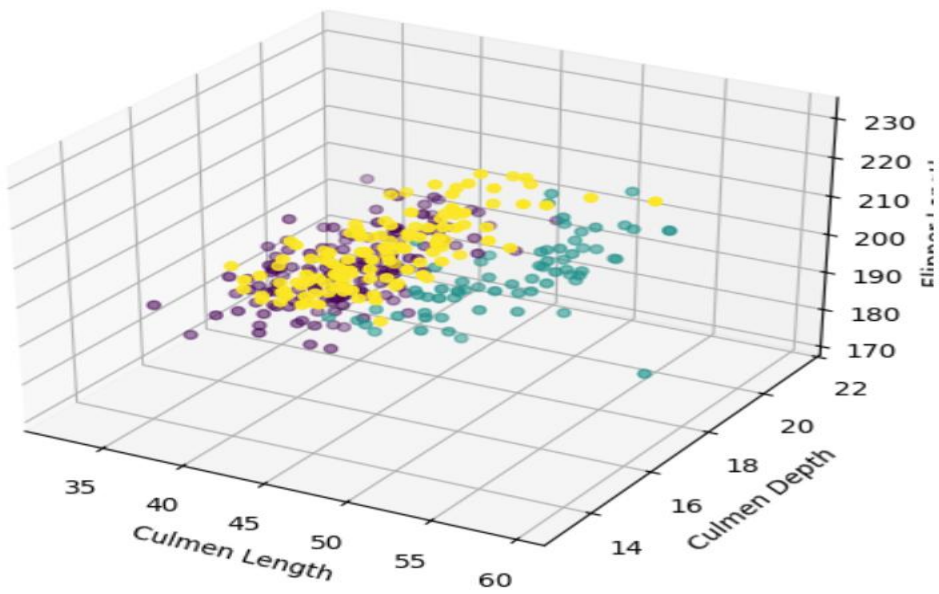
```
model.add(Dense(10, activation='relu', input_dim=3))
model.add(Dense(8, activation='relu'))
model.add(Dense(3, activation='softmax'))
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
history = model.fit(X_train, y_train,epochs=50,validation_split=0.2,verbose=1)
y_prob = model.predict(X_test)
y_pred = np.argmax(y_prob, axis=1)
accuracy = accuracy_score(y_test, y_pred)
print("\n RESULTS ")
print("Accuracy:", accuracy)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(X[:, 0],X[:, 1],X[:, 2],c=y)
ax.set_xlabel("Culmen Length")
ax.set_ylabel("Culmen Depth")
ax.set_zlabel("Flipper Length")
plt.title("3D Visualization of Penguin Classes")
plt.show()
sample = X_test[0].reshape(1, 3)
prediction = model.predict(sample)
predicted_class = np.argmax(prediction)
print("\n SAMPLE PREDICTION ")
print("Predicted Class:", predicted_class)
print("Actual Class:", y_test[0])
```

OUTPUT:

```
First 5 rows of dataset:
  species     island  culmen_length_mm  culmen_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen               39.1             18.7              181.0
1  Adelie  Torgersen               39.5             17.4              186.0
2  Adelie  Torgersen               40.3             18.0              195.0
3  Adelie  Torgersen                NaN              NaN                NaN
4  Adelie  Torgersen               36.7             19.3              193.0

   body_mass_g     sex
0       3750.0    MALE
1       3800.0  FEMALE
2       3250.0  FEMALE
3          NaN     NaN
4       3450.0  FEMALE
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 28ms/step - accuracy: 0.2227 - loss: 39.3342 - val_accuracy: 0.1455 - val_loss: 40.5301
Epoch 2/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2227 - loss: 35.1203 - val_accuracy: 0.1455 - val_loss: 35.9585
Epoch 3/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2227 - loss: 31.0599 - val_accuracy: 0.1455 - val_loss: 31.5875
Epoch 4/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2227 - loss: 27.1194 - val_accuracy: 0.1455 - val_loss: 27.3962
Epoch 5/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2227 - loss: 23.3783 - val_accuracy: 0.1455 - val_loss: 23.3360
Epoch 6/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2227 - loss: 19.7361 - val_accuracy: 0.1455 - val_loss: 19.4026
Epoch 7/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2227 - loss: 16.1571 - val_accuracy: 0.1455 - val_loss: 15.5893
Epoch 8/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2227 - loss: 12.7660 - val_accuracy: 0.1455 - val_loss: 11.8086
Epoch 9/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2227 - loss: 9.3288 - val_accuracy: 0.1455 - val_loss: 8.0888
Epoch 10/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2227 - loss: 6.0246 - val_accuracy: 0.1455 - val_loss: 4.4697
Epoch 11/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1864 - loss: 3.3785 - val_accuracy: 0.4545 - val_loss: 2.8521
Epoch 12/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.4136 - loss: 2.7758 - val_accuracy: 0.4727 - val_loss: 2.6559
Epoch 13/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.4182 - loss: 2.6127 - val_accuracy: 0.4727 - val_loss: 2.3030
Epoch 14/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.4182 - loss: 2.2292 - val_accuracy: 0.4727 - val_loss: 1.9038
Epoch 15/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.4136 - loss: 1.8415 - val_accuracy: 0.4545 - val_loss: 1.6441
Epoch 16/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.3091 - loss: 1.6047 - val_accuracy: 0.3091 - val_loss: 1.5021
Epoch 17/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1955 - loss: 1.4694 - val_accuracy: 0.2545 - val_loss: 1.3766
Epoch 18/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2227 - loss: 1.3543 - val_accuracy: 0.3636 - val_loss: 1.2568
Epoch 19/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.3273 - loss: 1.2737 - val_accuracy: 0.4000 - val_loss: 1.1762
```

```
Epoch 20/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.3636 - loss: 1.2214 - val_accuracy: 0.4364 - val_loss: 1.1273
Epoch 21/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3545 - loss: 1.1861 - val_accuracy: 0.4000 - val_loss: 1.0995
Epoch 22/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3364 - loss: 1.1578 - val_accuracy: 0.3818 - val_loss: 1.0809
Epoch 23/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.3227 - loss: 1.1400 - val_accuracy: 0.3818 - val_loss: 1.0648
Epoch 24/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2955 - loss: 1.1277 - val_accuracy: 0.3636 - val_loss: 1.0569
Epoch 25/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.2773 - loss: 1.1164 - val_accuracy: 0.3636 - val_loss: 1.0497
Epoch 26/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.2773 - loss: 1.1099 - val_accuracy: 0.3636 - val_loss: 1.0385
Epoch 27/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.2773 - loss: 1.1048 - val_accuracy: 0.3636 - val_loss: 1.0350
Epoch 28/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.2818 - loss: 1.0991 - val_accuracy: 0.3636 - val_loss: 1.0277
Epoch 29/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2682 - loss: 1.0942 - val_accuracy: 0.3636 - val_loss: 1.0276
Epoch 30/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.2636 - loss: 1.0914 - val_accuracy: 0.3636 - val_loss: 1.0231
Epoch 31/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2591 - loss: 1.0881 - val_accuracy: 0.3636 - val_loss: 1.0221
Epoch 32/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2545 - loss: 1.0848 - val_accuracy: 0.3636 - val_loss: 1.0198
Epoch 33/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2545 - loss: 1.0826 - val_accuracy: 0.3636 - val_loss: 1.0174
Epoch 34/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2500 - loss: 1.0806 - val_accuracy: 0.3455 - val_loss: 1.0185
Epoch 35/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2364 - loss: 1.0786 - val_accuracy: 0.3273 - val_loss: 1.0179
Epoch 36/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2409 - loss: 1.0769 - val_accuracy: 0.3636 - val_loss: 1.0110
Epoch 37/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2545 - loss: 1.0746 - val_accuracy: 0.3636 - val_loss: 1.0096
Epoch 38/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.2545 - loss: 1.0726 - val_accuracy: 0.3455 - val_loss: 1.0093
Epoch 39/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.2545 - loss: 1.0712 - val_accuracy: 0.3455 - val_loss: 1.0070
Epoch 40/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2455 - loss: 1.0696 - val_accuracy: 0.3091 - val_loss: 1.0097
Epoch 41/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.2136 - loss: 1.0694 - val_accuracy: 0.2727 - val_loss: 1.0116
Epoch 42/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2318 - loss: 1.0668 - val_accuracy: 0.3455 - val_loss: 1.0036
Epoch 43/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2409 - loss: 1.0658 - val_accuracy: 0.3273 - val_loss: 1.0039
Epoch 44/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.2409 - loss: 1.0629 - val_accuracy: 0.3455 - val_loss: 1.0005
Epoch 45/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.2545 - loss: 1.0618 - val_accuracy: 0.3455 - val_loss: 0.9982
Epoch 46/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2545 - loss: 1.0606 - val_accuracy: 0.3455 - val_loss: 0.9985
Epoch 47/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2318 - loss: 1.0592 - val_accuracy: 0.3273 - val_loss: 0.9996
Epoch 48/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2409 - loss: 1.0574 - val_accuracy: 0.3273 - val_loss: 0.9965
Epoch 49/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2545 - loss: 1.0578 - val_accuracy: 0.3636 - val_loss: 0.9919
Epoch 50/50
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2409 - loss: 1.0562 - val_accuracy: 0.2727 - val_loss: 0.9993
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step

 RESULTS
Accuracy: 0.3188405797101449

Confusion Matrix:
[[22 12  0]
 [11  0  0]
 [24  0  0]]

Classification Report:
              precision    recall  f1-score   support

           0       0.39      0.65      0.48        34
           1       0.00      0.00      0.00        11
           2       0.00      0.00      0.00        24

    accuracy                           0.32        69
   macro avg       0.13      0.22      0.16        69
weighted avg       0.19      0.32      0.24        69

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```






```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step

 SAMPLE PREDICTION
Predicted Class: 0
Actual Class: 2
```